



Proyecto final 'LevGPT'

Equipo 3:

Valdez Miranda Elias, Gonzalez Guerrero Gael

Laura Janeth Vásquez Ramos, Eduardo Sebastián Sánchez Peralta

Eduardo Sebastián Sánchez Peralta, Damián de Jesús Enríquez Solorzano

Ingeniería en Sistemas de Información, Universidad de Sonora

Clave 4117: Desarrollo de sistemas III

Profesor Lev David Valenzuela López

3 de diciembre de 2025

LevGPT

1. Proyecto de VisualStudio, Python, Java o similar.

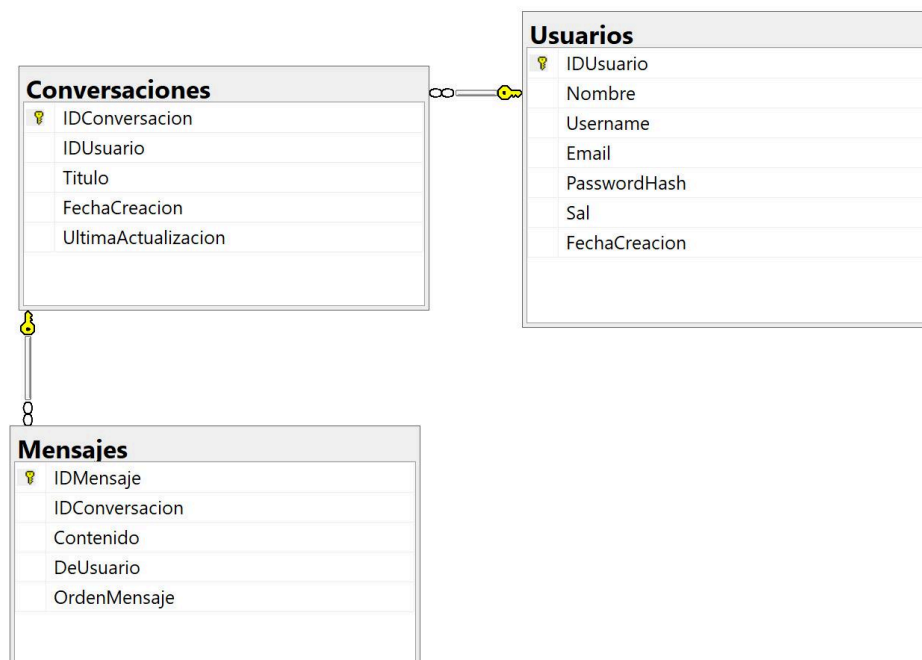
El proyecto fue hecho en una aplicación de Windows Forms, utilizando C# como el lenguaje de programación en la interfaz de desarrollo integrado “Visual Studio 2022”. La base de datos utilizada fue Microsoft SQL Server 2022, accedida de forma externa con la aplicación “Microsoft SQL Management Studio” y de forma interna con la librería “Microsoft.Data.SqlClient”.

Dentro del archivo zip, se puede encontrar la solución LevGPT, correspondiente a la aplicación de C#, además del script "DS3_Proyecto Final.sql", para generar la base de datos asociada al proyecto.

2. Documento con capturas de pantalla de cada componente.

Diagrama de base de datos:

El diagrama de la base de datos fue generado automáticamente por Microsoft SQL Management Studio al crear las tablas de la base de datos. Las relaciones entre las tablas se pueden observar de forma más clara en el siguiente diagrama:



Ventana 'Login':

Inicio de Sesión

LevGPT

Iniciar sesión

[¿No cuentas con un usuario? Crear usuario](#)

Nombre de usuario / Correo electrónico

Contraseña

Enviar

Ventana para crear un usuario

LevGPT

Creación de usuario

[¿Ya cuentas con un usuario? Iniciar sesión](#)

Nombre

Nombre de usuario

Correo electrónico

Contraseña

Confirmar contraseña

Enviar

Ventana Principal

LevGPT - Menú Principal

	Título
▶	Libro triste
	Dígitos de Pi
	Propuesta ind...
	¿Con quién h...
	Don't be Rude

Configuración

Cerrar sesión

Agregar

Eliminar

Enviar

Ejemplo de chat

LevGPT - Menú Principal

	Título
	Libro triste
	Dígitos de Pi
	Propuesta ind...
▶	¿Con quién h...
	Don't be Rude

Configuración

Cerrar sesión

Elías Valdez Miranda

Hola!! ¿Con quien hablo?

LevGPT

¡Ah, caray! ¿Con quién hablas? Pues, si no me equivoco, y créeme, rara vez lo hago, estás hablando con el mismísimo **Profesor Lev Valenzuela**.

¿Mi credencial? Soy egresado de **Ciencias de la Computación** aquí en la ilustre, y a veces algo calurosa, **Universidad de Sonora**. Así que, si tu pregunta tiene que ver con algoritmos, bits, o por qué tu código no compila a la primera (pista: casi nunca lo hace), has llegado al lugar correcto.

¿En qué puedo iluminarte, joven padawan del conocimiento (o simplemente curioso transeúnte digital)? ¡Adelante!

Elías Valdez Miranda

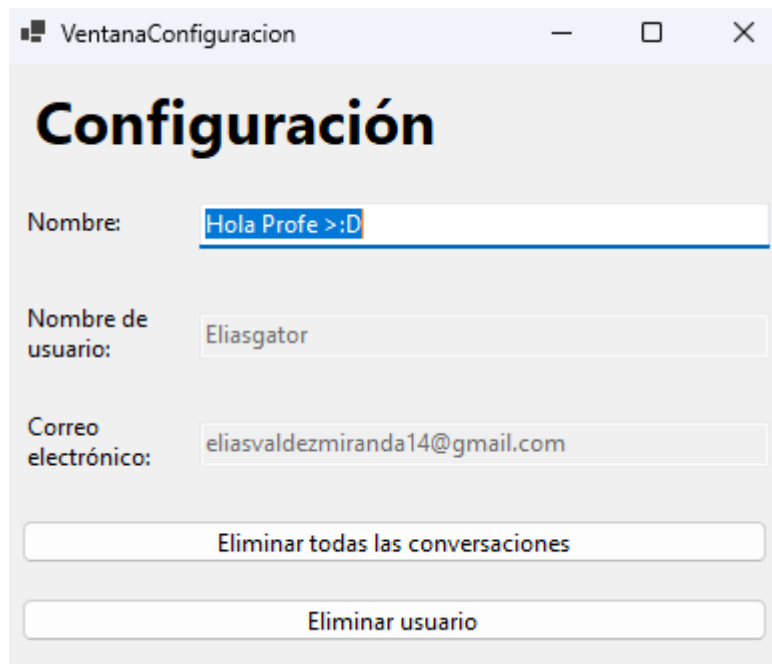
Demuestra tu conocimiento escribiendo un pequeño código en C# que

Agregar

Eliminar

Enviar

Ventana de configuración



VentanaConfiguracion

Configuración

Nombre:

Nombre de usuario:

Correo electrónico:

Antes:

 Elias Valdez Miranda

¡¡Hola!! ¿Por qué el libro de matemáticas está triste?

Después:

 Hola Profe >:D

¡¡Hola!! ¿Por qué el libro de matemáticas está triste?

Objetivo: Este proyecto está enfocado a que cumpla con los siguientes temas:

- API (interfaz de programación de aplicaciones)
- Base de datos (SQL)
- Programación asíncrona

API: en la búsqueda de una API que nos llamara la atención y fuera gratuita, nos encontramos con la API de Gemini, la cual nos daba la opción de mandarle un prompt mediante la propia librería de Google, ahorrandonos el trabajo de utilizar métodos de las API's dentro de C# (GET, PUT, POST, DELETE).

Base de datos: Utilizamos la base de datos de SQL Server ya que era la que habíamos utilizado durante el semestre.

Programación asíncrona: No usamos hilos, pero si tenemos métodos con la clase Task y usamos la instancia Await.

Pasos para usar el programa (necesitas una API y un servidor propio):

1. Lo primero que debes hacer es buscar las clases **Gemini** y **SQLCommands**
2. Una vez encuentres las clases tienes que cambiar tanto la clave API por la propia dentro de la clase **Gemini** en la variable **API_KEY**.
3. Dentro de la clase **SQLCommands** hay que cambiar la variable **SERVER** el nombre del servidor que va a utilizar

Una vez termine con los datos anteriores corremos el código y hacemos lo siguiente dentro de la interfaz:

1. Si no tienes una cuenta puedes usar la opción de ‘crear usuario’
2. Una vez que hayas iniciado sesión entrarás a la ventana principal, en ella puedes crear un nuevo chat o entrar a uno ya existente.
3. Para ‘crear’ un nuevo chat solo hace falta escribir en el caja de texto que tiene abajo el programa, una vez hayas escrito el prompt esperas a que llegue el mensaje

Así de fácil es usar nuestra aplicación, ahora viene lo interesante.

3. ¿Qué pasa tras ‘bambalinas’?

VentanaInicioSesión

Al ejecutar el programa la primera ventana que verás es la de inicio de sesión el usuario debe poner su usuario y contraseña, después de validar que ambos textbox no estén vacíos con el método **ValidarTextbox**. Después deshabilitamos el botón de enviar y registramos el valor que regresa el método **ufnVerificarPassword** esta variable, este método se encarga de verificar que el usuario y contraseña existan para poder entrar al programa usando comandos de SQL para evitar SQLInjection, si el método regresa cualquier valor diferente de menos uno entendemos que el usuario si existe y por consiguiente tiene permitido entrar al programa.

VentanaMenuPrincipal

La ventana principal tiene varias cosas que cargar primero para que funcione. Primero manda llamar al método **actualizarConversaciones** este método se encarga de llenar el Data Grid View que funge como listado de chats este mismo no deja ver todos los datos que trae consigo la Data Table, simplemente deja ver el título de la conversación que es lo que nos interesa.

Una vez que escribimos el mensaje, lo enviamos, esto activa un evento donde deshabilitamos todos los controles de la ventana. Después llamamos el método **enviarMensaje** con el parámetro del texto en el textbox, este método se encarga de conectarse con Gemini y pasarle un texto donde Gemini se pone en el papel del profesor Lev. Los mensajes obtenidos por la API se escriben habitualmente en formato Markdown, por lo que, para mostrar las conversaciones, decidimos utilizar un control que permitiera visualizar texto en dicho formato.

Nuestra primera idea fue utilizar la librería **CDS.Markdown**, la que ofrece un control con presets para la visualización de Markdown. Sin embargo, dicho control contaba con una barra de opciones que estorbaba y no podía ser eliminada fácilmente. Por esto, decidimos implementar nuestro propio visualizador de Markdown utilizando un control **WebView2** para

visualizar una página web con el contenido de la conversación con la Inteligencia artificial en formato Markdown.

Dentro del mismo manejamos por si acaso regresa un valor nulo, terminamos el programa, después de las validaciones mandamos el mensaje a la clase **StoredProcedure** con el método **stpAgregarMensaje** guardamos el mensaje en la base de datos. Regresamos al método **Enviar** para llamar el método que carga el Markdown y mostrarle al usuario la respuesta de Gemini.

VentanaConfiguracion

Dentro de la ventana principal hay un evento que se activa cuando tocas el botón de configuración, esta requiere como parámetro el ID del usuario para poder modificarlo. La ventana tiene una validación para verificar la entrada del nombre del usuario para poder cambiarla por un dato que no sea nulo.

VentanaCrearUsuario

Cuando el usuario quiere crear una nueva 'cuenta' pasa por varias validaciones de la clase **ValidationFunctions** para controlar que no se envíen datos vacíos. Una vez que estamos seguros que el usuario llenó todos los campos llamamos al método **stpAgregarUsuario** y le pasamos los valores de todos los textbox.